HARNESS LAB · MÓDULO 2 · AULA 2.4

Como não quebrar seu fluxo *com hooks*

Hook quebrado não avisa. Ele libera. Por mais determinístico que seja, um hook é código, e código tem bug. A correção de todos os tropeços é a mesma: testar antes de confiar.

`onFailure` `stop_hook_active` `timeout` `async` `asyncRewake` `CLAUDE_PROJECT_DIR` `chmod +x` `testa-hooks.sh`

VOCÊ SAI COM

O `testes/testa-hooks.sh` no seu projeto: confere cada hook ligado no settings, roda um caso que barra e um que passa para cada regra, e reprova na hora o hook que ia falhar calado. As travas com `"onFailure": "block"`.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A última aula do Módulo 2 é uma revisão com um aviso: não crie hooks demais, a ponto de virar um sistema complexo, e cuide para que eles não quebrem o seu fluxo. Hook é determinístico, mas você pode escrever bug nele.

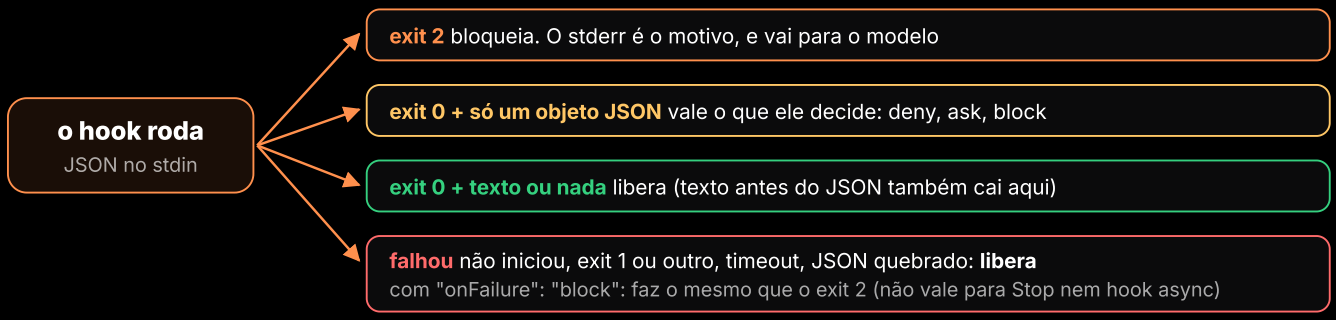
Hook quebrado não avisa, ele libera. E, para o hook que existe para ajudar, é isso que você quer: se ele falhou, que diga ao modelo que falhou e deixe seguir. Senão vira loop: o hook falha, falha, falha, e o modelo acha que o erro é dele. No código do hook, isso é um `try/catch` que separa a falha de validação (barra) da exceção, do erro 500 (deixa passar).

Dois tropeços que acontecem de verdade: **deixar passar** quando não devia e **barrar demais**. Os dois pedem medir e testar. Regex não é muro: hook, permissões do Claude Code e teste, juntos. Quanto mais hooks, mais atenção.

Mais dois que custam caro. **O hook que não deixa parar**: um hook no `Stop` que não deixa a tarefa terminar faz o agente rodar em loop e consumir o que não precisava. **O hook que trava a sessão**: hook normal é síncrono, a sessão espera ele terminar. Se a suíte de testes leva duas horas, rodar tudo a cada edição mata a sessão. Rode só o teste do que foi editado, ou do grupo.

Os **quatro jeitos de quebrar em silêncio**: permissão de execução (sem ela o script não inicia), caminho que muda (você moveu o script e esqueceu o settings), dependência que falta (o `jq`) e stdout que vira contexto. E o fechamento: **teste antes de confiar**. Peça ao Claude para testar cada hook com vários cenários, crie um `testa-hooks` e rode. Módulo 2 fechado: a camada de trava do harness está de pé.

COMO O CLAUDE CODE LÊ A RESPOSTA DE UM HOOK



Hook quebrado libera

Falhou, deixa seguir e avisa o modelo que falhou. Senão o modelo acha que o erro é dele e entra em loop. No código: `try/catch`, e erro 500 deixa passar.

Regex não é muro

Hook lê texto, não intenção. Deixa passar o que não previu e barra o que só parece perigoso. Teste os dois lados.

Stop sem freio é loop

O hook que não deixa a tarefa terminar é o pior caso: loop infinito, consumindo à toa. O freio está na seção 2.

Síncrono segura a sessão

A sessão espera o hook terminar. Suíte de duas horas mata a sessão: rode só o teste do que foi editado, ou do grupo.

> *"On most events, when a hook fails or times out, Claude Code still carries out the action. A policy hook with a wrong path or a crashing script then lets everything through."*
>
> Documentação oficial do Claude Code · Hooks reference · Block the action when a hook fails

2 O que a aula *não teve tempo de dizer*

ALÉM DA AULA · LIBERAR OU BARRAR QUANDO QUEBRA: DEPENDE DO HOOK

"Hook quebrado tem que passar para frente" vale para o hook que ajuda: lint, teste, log, contexto. Para a trava, que existe para impedir alguma coisa, liberar quando quebra é exatamente o estrago. Nela, ponha `"onFailure": "block"` (Claude Code 2.1.295 ou mais novo): a falha passa a fazer o mesmo que um exit 2. Não tem efeito em `Stop`, `SubagentStop`, `TaskCompleted`, `TeammateIdle` nem em hook `async`. E pense duas vezes na trava que depende de rede, como o `guard-jev.py` da aula 2.3: com `"block"`, a API fora do ar barra todo comando.

Teste feito para este material, no Claude Code 2.1.316, com o `bloqueia-perigoso.sh` sem `chmod +x`: sem `onFailure`, o `echo` pedido rodou. Com `"block"`, **todo** comando Bash foi barrado com `Permission denied`, e o modelo não tinha como consertar: o conserto (`chmod +x`) é um comando Bash. Quem conserta é você, no seu terminal.

Fonte: Hooks reference, "Block the action when a hook fails".

ALÉM DA AULA · UM ECHO DE DEPURAÇÃO APAGA A DECISÃO

O Claude Code só lê o JSON de decisão se o stdout for um objeto JSON e mais nada. Um `echo "debug: $cmd"` antes do `permissionDecision: "deny"` faz a saída começar com `d`: vira texto puro, exit 0, libera. Sem aviso nenhum. Teste feito para este material, no Claude Code 2.1.316: com o `modelos/quebrados/eco-no-stdout`, o `cat .env` rodou e o segredo apareceu na conversa. Com o hook certo, foi barrado. Depuração vai para o stderr (`>&2`) ou para um arquivo de log.

No `SessionStart` e no `UserPromptSubmit` o problema é o contrário: o texto puro do stdout entra como contexto do modelo. É o "stdout que vira contexto" da aula.

Fonte: Hooks reference, "Exit code 0".

ALÉM DA AULA · EXIT 1 NÃO BARRA (E COM ONFAILURE BARRA TUDO)

O `exit 1` é a falha de todo script Unix, e no Claude Code é um erro que não bloqueia: você vê `hook error` na conversa e a ação segue. Trava barra com `exit 2` ou com JSON de decisão e `exit 0`. Com `"onFailure": "block"`, qualquer código diferente de 0 e 2 vira falha e barra, mesmo que o hook tenha impresso um JSON que libera. O `try/catch` da aula decide isso: no `catch` da trava, `exit 2`; no do hook que ajuda, `exit 0` com o aviso.

Fonte: Hooks reference, "Exit code output" e "Block the action when a hook fails".

ALÉM DA AULA · O LIMITE DE 8 DO STOP ZERA A CADA FERRAMENTA

O Claude Code encerra a resposta depois de 8 continuações seguidas causadas por Stop hooks. Mas a contagem zera toda vez que o modelo chama uma ferramenta. Um Stop que diz "rode os testes de novo" faz o modelo rodar os testes, que é uma ferramenta: a contagem volta a zero, e o limite nunca chega. O freio de verdade é o seu: `stop_hook_active` vem `true` quando ele já está continuando por causa de um Stop hook. Nesse caso, `exit 0`. O `testa-hooks.sh` roda cada Stop do seu settings com esse campo em `true` e reprova quem devolve exit 2 de novo.

Fonte: Hooks reference, "Stop input". A variável `CLAUDE_CODE_STOP_HOOK_BLOCK_CAP` muda o limite.

ALÉM DA AULA · SEGUNDO PLANO: O QUE MUDA COM ASYNC

Com `"async": true`, o Claude Code não espera e não aplica o `timeout`. O resultado chega no turno seguinte e o hook não consegue barrar nada. Com `"asyncRewake": true`, também em segundo plano, um exit 2 acorda o modelo na hora com o stderr, e o `timeout` continua valendo. No `claude -p`, hook em segundo plano que ainda está rodando no fim é cancelado. Modelo pronto: `modelos/suite-completa.sh` e `modelos/settings.sem-quebrar.json`.

Para achar o hook lento: `claude --debug` grava em `~/.claude/debug/<session-id>.txt` (não no terminal). Hook de comando em evento de ferramenta, `UserPromptSubmit`, `SessionStart` ou `Stop` deixa uma linha com `Hooks:` no começo e a duração no fim (Claude Code 2.1.296 ou mais novo). Em `Notification`, `SessionEnd` e `PreCompact` a linha sai sem duração.

Fonte: Hooks reference, "Run hooks in the background" e "Debug hooks".

ALÉM DA AULA · QUANDO UM HOOK PRENDE A SESSÃO

Para abrir uma sessão com todos os hooks desligados, sem mexer em arquivo nenhum: `claude --settings '{"disableAllHooks": true}'` (o comando está no passo 9). Não dá para desligar um hook só e manter os outros: tire ele do settings. Edição direta no settings costuma valer na hora, sem reiniciar a sessão. E o `/hooks` mostra o que está ligado e de qual arquivo veio.

Fonte: Hooks reference, "Disable or remove hooks"; Hooks guide, "Hook not firing".

3 Instale e teste *no seu repositório*

O que vai para o seu projeto: `testes/testa-hooks.sh` (o motor, não precisa editar) e `testes/casos-hooks.sh` (os casos, esse é seu). Os casos que vêm prontos cobrem os hooks das aulas 2.2 e 2.3. Hook que você não tem é pulado. Precisa de `jq`, o mesmo dos hooks.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale o teste no seu projeto.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 2.4
```

3 Rode nos seus hooks.

Terminal · na raiz do seu projeto — shell

```
bash testes/testa-hooks.sh
```

Primeiro ele confere o settings: cada hook ligado existe, tem `chmod +x`, usa `"$CLAUDE_PROJECT_DIR"`, a trava tem `onFailure`, o Stop respeita o `stop_hook_active`. Depois roda os casos. Com os hooks da aula 2.3 instalados:

saída real, encurtada — ✓ passou · ! aviso · ✗ reprovou · - pulado

```
settings (hooks ligados neste projeto)
  ✓  PreToolUse (Bash) guard-destructive.py        existe e inicia
  !  PreToolUse (Bash) guard-jev.py                sem onFailure: se quebrar, a ação segue
  ✓  PreToolUse (Read|Edit|Write) protege-env.sh   existe e inicia
casos (hooks de .claude/hooks)
  ✓  migrate:fresh via sail                        bloqueia
  ✓  prisma migrate reset                          bloqueia
  ✓  migrate:status                                libera
  ✓  Read .env.example                             libera
  -  quality-gate.sh não existe                    casos pulados

23 passaram, 0 falharam, 1 avisos, 22 pulados.
```

4 Veja o teste pegar um hook quebrado.

Os hooks quebrados de propósito da aula ficam no repositório do curso, não no seu projeto. Rode o teste contra cada um. O `|| echo` está ali porque o teste sai com código 1 quando reprova, que é o

esperado:

Terminal · na raiz do seu projeto — shell

```
M=~/harness-lab-materiais/modulo-2-trava
Q=$(echo $M/aula-2.4-*/modelos/quebrados)
bash testes/testa-hooks.sh "$Q/campo-errado" || echo "pegou"
bash testes/testa-hooks.sh "$Q/demais" || echo "pegou"
bash testes/testa-hooks.sh "$Q/eco-no-stdout" || echo "pegou"
```

`campo-errado` lê `.tool_input.cmd` no lugar de `.tool_input.command`: tudo vem `libera`, calado. `demais` barra `migrate:status` e `cat .env.example`: são os casos "tem que passar" que reprovam. `eco-no-stdout` é o hook certo com um `echo` a mais: a decisão some.

5 Agora tire o chmod +x de uma cópia dos seus.

Terminal · na raiz do seu projeto — shell

```
rm -rf /tmp/hooks-sem-x
cp -r .claude/hooks /tmp/hooks-sem-x
chmod -x /tmp/hooks-sem-x/*.sh
bash testes/testa-hooks.sh /tmp/hooks-sem-x || echo "pegou"
```

Os casos dos hooks `.sh` mostram `falhou(exit 126)`: o script não iniciou. No Claude Code, sem `onFailure`, isso seria um aviso de `hook error` e a ação rodando. O `guard-destructive.py` segue passando: o settings chama ele com `python3`, que não precisa do `chmod +x`, e o teste faz o mesmo.

6 Escreva os casos do seu projeto.

Para cada regra, um caso que tem que barrar e um parecido que tem que passar. O segundo é o que pega o "barrou demais". Cada linha de `testes/casos-hooks.sh` tem este formato:

testes/casos-hooks.sh — instalado pelo instalar.sh · não copie do PDF

```
# caso "HOOK [args]" "descrição" JSON ESPERADO
caso guard-destructive.py "migrate:fresh" \
  "$(bash_json 'php artisan migrate:fresh --seed')" bloqueia
caso guard-destructive.py "migrate:status" \
  "$(bash_json 'php artisan migrate:status')" libera
caso protege-env.sh "Read .env.example" "$(arq_json Read .env.example)" libera
caso "quality-gate.sh stop" "Stop já continuando" "$(stop_json true)" libera
```

Na aula, o caminho é pedir ao Claude. O pedido completo, que está em `modelos/pedir-o-teste.md`:

> **O PEDIDO**
>
> "Leia os hooks em .claude/hooks e o .claude/settings.json. Para cada regra de cada hook, escreva em testes/casos-hooks.sh um caso que tem que barrar e um caso parecido que tem que passar, usando os comandos reais deste projeto. Rode bash testes/testa-hooks.sh e me mostre o resultado. Não mude os hooks: se um caso falhar, me diga qual regra está errada."

Ou a versão curta:

Dentro do Claude Code — sessão

```
> Escreva os casos de testes/casos-hooks.sh sem mudar os hooks
```

O "não mude os hooks" importa: se ele pode mexer no hook e no teste ao mesmo tempo, ele faz os dois concordarem. Você quer saber onde a regra falha. Outra stack? Os casos do `guard-destructive.py` já têm o par para Prisma (`migrate reset` e `migrate status`) e Rails (`db:reset` e `db:migrate`).

7 Confira o onFailure de cada trava.

O teste avisa `sem onFailure` em cada `PreToolUse` sem o campo. Decida um por um. Trava com regra local (`guard-destructive.py`, `protege-env.sh`): `"block"`. O `guard-jev.py` da aula 2.3 fica sem, de propósito: ele chama a API do Jev, e com `"block"` a API fora do ar barraria todo comando. Com a 2.3 instalada, as duas travas locais já vêm com `"block"` e só o `guard-jev.py` aparece sem. Veja como estão os seus:

Terminal · na raiz do seu projeto — shell

```
F=.claude/settings.json
jq -c '.hooks.PreToolUse[]?.hooks[]? | {command, onFailure}' $F
```

Tem uma trava local sua sem o campo? Peça, rode o teste de novo e confira com a sessão aberta: o `/hooks` lista cada hook, o evento e o arquivo de onde veio.

Dentro do Claude Code — sessão

```
> Ponha "onFailure": "block" nas travas locais que estão sem
/hooks
```

8 Commite os hooks e o teste juntos.

Terminal · na raiz do seu projeto — shell

```
git add .claude/settings.json .claude/hooks testes
git commit -m "hooks com teste"
```

O time inteiro recebe os dois. Rode o `testa-hooks.sh` antes de cada mudança num hook.

9 Se um hook prender a sessão.

Abra uma sessão com todos os hooks desligados, conserte com calma, e rode o teste antes de religar:

Terminal · na raiz do seu projeto — claude

```
claude --settings '{"disableAllHooks": true}'
```

4 Arquivos que *reforçam a aula*

`testes/testa-hooks.sh`

A frase "teste antes de confiar" virada em comando. Lê a resposta do hook com as mesmas regras do Claude Code, inclusive as que liberam calado: texto antes do JSON, exit 1, JSON quebrado.

No repositório: modulo-2-trava/aula-2.4-como-nao-quebrar-seu-fluxo-com-hooks/projeto/testes/testa-hooks.sh

`testes/casos-hooks.sh`

Um caso que barra e um que passa para cada regra, para o `guard-destructive.py`, o `protege-env.sh` e o `quality-gate.sh` (o caso que barra cria um `.py` com erro de sintaxe e apaga depois), com o caso real do `grep` com `|` no padrão. É o arquivo que você adapta.

No repositório: modulo-2-trava/aula-2.4-como-nao-quebrar-seu-fluxo-com-hooks/projeto/testes/casos-hooks.sh

`modelos/quebrados/`

Os defeitos da aula, de propósito: campo errado, barrar demais, echo no stdout, Stop em loop e hook lento. Prova que o seu teste pega cada um.

No repositório: modulo-2-trava/aula-2.4-como-nao-quebrar-seu-fluxo-com-hooks/modelos/quebrados/

`modelos/prova-no-stop.sh`

O Stop certo: roda os testes antes de encerrar e tem o freio do `stop_hook_active` na primeira linha útil. Se você já usa o `quality-gate.sh stop` da 2.2, não ligue os dois.

No repositório: modulo-2-trava/aula-2.4-como-nao-quebrar-seu-fluxo-com-hooks/modelos/prova-no-stop.sh

`modelos/settings.sem-quebrar.json`

Os três ajustes num lugar: `onFailure` na trava, `timeout` e `statusMessage` no hook de edição, `asyncRewake` na suíte completa (`modelos/suite-completa.sh`).

No repositório: modulo-2-trava/aula-2.4-como-nao-quebrar-seu-fluxo-com-hooks/modelos/settings.sem-quebrar.json

`modelos/pedir-o-teste.md`

O pedido para o Claude escrever os casos do seu projeto, e por que cada parte dele muda o resultado. Tem a versão para quem não instalou o `testa-hooks.sh`.

No repositório: modulo-2-trava/aula-2.4-como-nao-quebrar-seu-fluxo-com-hooks/modelos/pedir-o-teste.md

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`PreToolUse:Bash hook error` e o comando rodou	O script não iniciou: caminho errado ou sem `chmod +x`. Sem `onFailure`, libera	`chmod +x`, caminho com `"$CLAUDE_PROJECT_DIR"`, `"onFailure": "block"` na trava
Todo comando Bash barrado com `blocking because onFailure is "block"`	A trava com `onFailure` quebrou. O modelo não conserta: o conserto também é Bash	Conserte no seu terminal e rode `bash testes/testa-hooks.sh`
O hook imprime deny e a ação roda, sem erro	Texto no stdout antes do JSON: vira texto puro	Depuração no stderr ou em log. O teste mostra `veio libera`

SINTOMA	CAUSA	CORREÇÃO
O hook sai com 1 e não barra	`exit 1` é erro que não bloqueia	`exit 2`, ou JSON de decisão com `exit 0`
O agente não para de voltar a trabalhar	Stop sem o freio. O limite de 8 zera a cada ferramenta	Primeira linha: `stop_hook_active true` sai com 0
A sessão fica parada a cada edição	Hook síncrono rodando a suíte inteira	Só o teste do arquivo, `timeout`, ou `asyncRewake`
`<hook> não existe` e, na mesma linha, `casos pulados`	O nome nos casos não é o do seu script	Ajuste o nome em `testes/casos-hooks.sh`
No macOS: `sem timeout no sistema`	O macOS não tem `timeout`	`brew install coreutils` (o teste usa o `gtimeout`)
A permissão `deny` não pegou `git push origin main --force`	Regra de permissão casa o começo do comando como está escrito	Permissão para o simples, hook para o resto, e teste nos dois

6 Você sai *com*

- [ ] `bash testes/testa-hooks.sh` termina com `0 falharam` no seu projeto.
- [ ] Para cada regra das suas travas, um caso que barra e um parecido que passa.
- [ ] O teste reprovou os quebrados da aula: campo errado, barrar demais, echo no stdout, sem `chmod +x`.
- [ ] As travas locais de `PreToolUse` com `"onFailure": "block"`, conferidas no `/hooks`.
- [ ] Cada Stop do settings respeita o `stop_hook_active` (a parte de settings do teste mostra).
- [ ] Nenhum hook síncrono roda a suíte inteira a cada edição.
- [ ] Hooks e teste no mesmo commit.

PROMETIDO NA AULA

- ✓ "Cria um testa-hooks e executa esse comando": `testes/testa-hooks.sh`, instalado pelo `instalar.sh 2.4` (passos 2 e 3).
- ✓ "Pede pro Claude testar se esse hook está funcionando com vários cenários": o pedido em `modelos/pedir-o-teste.md` e no passo 6.
- ✓ "Todos os hooks que você fez devem estar testados": os casos de `testes/casos-hooks.sh` cobrem os hooks das aulas 2.2 e 2.3.
- ✓ "Rodar teste somente do que foi editado": o `quality-gate.sh` da aula 2.2; para a suíte inteira, `modelos/suite-completa.sh` com `asyncRewake`.

7 Glossário

Falhar aberto, falhar fechado	Aberto: o hook quebra e a ação segue (o padrão). Fechado: o hook quebra e a ação é barrada.
onFailure	Campo do hook no settings. Com `"block"`, a falha faz o mesmo que um exit 2. Claude Code 2.1.295 ou mais novo.
stop_hook_active	Campo do JSON do Stop. `true` quando o agente já está continuando por causa de um Stop hook.
timeout	Segundos até o hook ser cancelado. Padrão de 600 para hook de comando; 30 no `UserPromptSubmit`.
async, asyncRewake	Rodam o hook em segundo plano. O `asyncRewake` acorda o modelo num exit 2 e mantém o `timeout`.
CLAUDE_PROJECT_DIR	A raiz do projeto. Não muda quando o modelo dá `cd`, por isso vai no caminho do hook.
exit 126	O shell achou o script mas não pôde executar: falta o `chmod +x`.
disableAllHooks	Setting que desliga todos os hooks, sem apagar a configuração.

8 Para ir *além*

- Hooks reference: o que conta como falha, o `onFailure`, a tabela de stdout por exit code, o Stop com o limite de 8, async e o log de debug.
- Hooks guide, Limitations and troubleshooting: hook que não dispara, Stop que não para de bloquear, erro de JSON.
- Permissions: as regras `allow`, `ask` e `deny` e como o `*` casa o comando. A parte dura da trava, sem regex.
- Debug your config: qual settings ganhou, o `/doctor` e como achar o que está carregado.

Próxima: *Aula 3.1 · CLAUDE.md enxuto*

A trava está de pé e testada. Agora falta ensinar o agente a trabalhar do seu jeito, começando pelo arquivo que todo mundo enche demais.